



UNIVERSIDAD
NACIONAL
DE COLOMBIA



Sesión 4: Funciones como valores

Semillero en Ciencias de la Computación

Tomás Rodríguez Santiago López

Mayo 22, 2026



1. Funciones como valor de parámetro
2. Funciones como valor de retorno
3. Conclusiones

El Secreto de la Mantenibilidad

Tratar las funciones como **valores** proporciona la flexibilidad necesaria para elaborar código mantenible y altamente cohesivo.

¿Qué significa esto? Que una función no es solo un bloque estático de instrucciones. Es un dato más que podemos asignar a variables, **pasar como argumento** a otras funciones o **retornarlo** como resultado.

Nuestra Historia de Hoy: Construiremos el motor de reglas de un Juego de Palabras (*Ranking de palabras*) para ver esto en acción.



1. Funciones como valor de parámetro

Composición:

$$(f \circ g)(x) = f(g(x))$$

En Clojure:

`(comp f g)`

Ejemplo:

```
(require '[clojure.string :as str])  
(def procesar-texto (comp str/upper-case str/trim))  
  
(procesar-texto "  hola clojure  ")  
;; => "HOLA CLOJURE"
```

- De acuerdo con el ejemplo, ¿Qué hace la función **trim**?

1a. Ordenar por puntaje (Fijo)



Requerimiento del Cliente (1a)

"Necesito calcular el puntaje base de una palabra. El puntaje es la cantidad de letras que quedan si eliminamos todas las letras 'a'. Luego, hay que ordenar las palabras según este puntaje."

Ejercicio: Implemente la función de costo

Hint: ¿Qué hace la función `replace` en `clojure`?

<https://onecompiler.com/clojure>

1a. Ordenar por puntaje (Fijo)

Requerimiento del Cliente (1a)

"Necesito calcular el puntaje base de una palabra. El puntaje es la cantidad de letras que quedan si eliminamos todas las letras 'a'. Luego, hay que ordenar las palabras según este puntaje."

Ejercicio: Implemente la función de costo

```
(require '[clojure.string :as str])  
(def words '("java" "ada" "scala" "haskell" "rust"))  
  
(def score (comp count #(str/replace % "a" "")))
```

1a. Ordenar por puntaje (Fijo)



Requerimiento del Cliente (1a)

"Necesito calcular el puntaje base de una palabra. El puntaje es la cantidad de letras que quedan si eliminamos todas las letras 'a'. Luego, hay que ordenar las palabras según este puntaje."

Ejercicio: Implemente la función de ordenamiento

Hint: ¿Qué hace la función sort-by?

1a. Ordenar por puntaje (Fijo)

Requerimiento del Cliente (1a)

"Necesito calcular el puntaje base de una palabra. El puntaje es la cantidad de letras que quedan si eliminamos todas las letras 'a'. Luego, hay que ordenar las palabras según este puntaje."

Implementamos la regla usando composición y ordenamos de forma directa:

```
(require '[clojure.string :as str])  
(def words '("java" "ada" "scala" "haskell" "rust"))  
  
(def score (comp count #(str/replace % "a" "")))  
  
(defn ranked-words-naive [words]  
  (reverse (sort-by score words)))
```

1b. Un pequeño cambio



Requerimiento del Cliente (1b)

"Las reglas cambiaron. Ahora sumamos 5 puntos de bonus si la palabra tiene 'c', y restamos 7 si tiene 's'. ¡Y no quiero que reescriban todo el sistema de ordenamiento por este cambio!"

Ejercicio: Implemente el requerimiento.

Hint: ¿Qué hace la función includes? en clojure?

1b. Un pequeño cambio

Requerimiento del Cliente (1b)

"Las reglas cambiaron. Ahora sumamos 5 puntos de bonus si la palabra tiene 'c', y restamos 7 si tiene 's'. ¡Y no quiero que reescriban todo el sistema de ordenamiento por este cambio!"

Solución:

```
(defn ranked-words [word-score words]
  (->> words
    (sort-by word-score)
    reverse))
```

1. ¿Qué hace `- >` en clojure?
2. ¿Qué hace `- >>` en clojure?

¿Cómo funciona?

Inserta el resultado de cada expresión como el **primer argumento** de la siguiente función.

Caso de uso ideal:

Transformación de estructuras de datos como mapas o registros (usando funciones como `assoc`, `dissoc`, `update`).

Ejemplo:

```
(assoc (conj (dissoc {:a 1 :b 2} :a) [:c 3]) :d 4)
;; => {:b 2, :c 3, :d 4}
```

```
(-> {:a 1 :b 2}
    (dissoc :a)
    (conj [:c 3])
    (assoc :d 4))
;; => {:b 2, :c 3, :d 4}
```

¿Cómo funciona?

Inserta el resultado de cada expresión como el **último argumento** de la siguiente función.

Caso de uso ideal:

Procesamiento de secuencias y colecciones (usando funciones como map, filter, reduce).

Ejemplo:

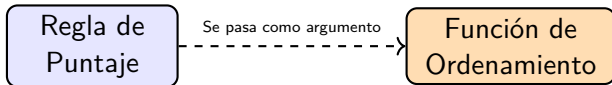
```
(reduce + (map inc (filter even? [1 2 3 4 5 6])))  
;; => 15
```

```
(->> [1 2 3 4 5 6]  
      (filter even?) ;; [2 4 6]  
      (map inc)       ;; [3 5 7]  
      (reduce +))     ;; => 15
```

Problema

Si nuestra lógica de negocio cambia, no queremos reescribir nuestra función de ordenamiento (`ranked-words`).

Solución: Si tratamos a la función de calcular puntaje como un **valor**, podemos pasarla por parámetro. Así, la función que ordena delega la lógica de negocio a quien la llama.





Definición

Una *función de orden superior* es una función que realiza al menos una de estas dos acciones: recibe otra función como parámetro (argumento), o devuelve una función como resultado

Podemos ir más allá, dejando funciones pequeñas con responsabilidades pequeñas.

Código completo:

```
(defn bonus [word] (if (str/includes? word "c") 5 0))  
(defn penalty [word] (if (str/includes? word "s") 7 0))  
  
(defn total-score [word]  
  (- (+ (score word) (bonus word)) (penalty word)))  
  
(defn ranked-words [word-score words]  
  (->> words  
    (sort-by word-score)  
    reverse))
```


¡El cliente contraataca!

Requerimiento del Cliente (1c)

"Para el panel de administración, no necesito ordenar las palabras. Solo necesito ver una lista cruda con los puntajes calculados de las palabras actuales."

Ejercicio: Implementar el requerimiento.

Requerimiento del Cliente (1c)

"Para el panel de administración, no necesito ordenar las palabras. Solo necesito ver una lista cruda con los puntajes calculados de las palabras actuales."

Solución:

```
(defn word-scores [word-score words]  
  (map word-score words))
```



2. Funciones como valor de retorno

2d. Filtrar por Threshold

Requerimientos del Cliente (2d y 2e)

"Quiero un reporte que traiga solo las palabras con un puntaje mayor a 1"

Ejercicio: Implemente este requerimiento.

2d. Filtrar por Threshold



Requerimientos del Cliente (2d y 2e)

"Quiero un reporte que traiga solo las palabras con un puntaje mayor a x"

Solución:

```
(defn highest-words-naive [word-score words threshold]  
  (filter #( > (word-score %) threshold) words))
```

Cliente

"La API es muy incómoda de usar!!! Tengo hacer código repetitivo para un cambio muy mínimo!!"

Por ejemplo:

```
(defn highest-words-naive [word-score words threshold]
  (filter #( > (word-score %) threshold) words))

;; El desarrollador cliente tiene que repetir argumentos:
(highest-words-naive total-score words 1)
(highest-words-naive total-score words 3)
(highest-words-naive total-score words 5)
```

Cliente

"La API es muy incómoda de usar!!! Tengo hacer código repetitivo para un cambio muy mínimo!!"

El Problema

Exigir pasar la lógica de puntaje y la lista de palabras **cada vez** que solo queremos cambiar el tresshold genera redundancia y una mala API.



Cliente

"La API es muy incómoda de usar!!! Tengo hacer código repetitivo para un cambio muy mínimo!!"

Ejercicio: Salve su empleo.

Cliente

"La API es muy incómoda de usar!!! Tengo hacer código repetitivo para un cambio muy mínimo!!"

Solución:

```
(defn highest-words [word-score words]
  (fn [threshold]
    (filter #( > (word-score %) threshold) words)))
```

¿Qué está retornando esta función?

Código completo:

```
(defn highest-words [word-score words]
  (fn [threshold]
    (filter #( > (word-score %) threshold) words)))

;; 'higher-than' es ahora una funcion!
(def higher-than
  (highest-words total-score words))

;; Ejecucion limpia y hermosa para el cliente final:
user=> (higher-than 0)
user=> (higher-than 5)
```

Ya sabemos como recibir funciones por parámetro, pero el paradigma funcional también permite **retornar** funciones como resultado.

Definición

Closure es una función que “cierra” o captura las variables del contexto inmediato en el momento que fue creada.

- ¿Qué closure vemos en la última solución?

El **Currying** (o aplicación parcial) es una técnica donde una función que toma múltiples argumentos se transforma en una serie de funciones que toman **un solo argumento**.

¿Para qué sirve en la práctica?

1. **Configuración previa:** “Pre-cargamos” una función con los parámetros pesados o estáticos (como la lista de palabras y las reglas de negocio).
2. **Retorno diferido:** Retornamos una nueva función ligera que solo pide el último dato que falta (el umbral).
3. **Limpia la API:** El usuario final de nuestro código tiene una experiencia mucho más cómoda.

¿Qué pasaría si lo único que quiero “precargar” es la lista de palabras a procesar?

Pruebe el siguiente código:

```
(def suma5 (partial + 5))  
(suma5 10)
```

¿Qué cree que hace la función *partial* aquí?

Definición:

partial toma una función f y menos de los argumentos normales para f y retorna una función que recibe el número restante de argumentos.

Ejercicio: Refactorice *highest-words* usando *partial*



3. Conclusiones

1. Versatilidad

Al pasar funciones como parámetros (`word-score`), separamos el “¿Qué hacer?” (nivel micro) del “¿Cómo orquestarlo?” (nivel macro).

2. APIs Ergonómicas

Al retornar funciones (Currying), creamos herramientas pre-configuradas que ocultan la complejidad al consumidor de nuestro código.

3. La fuerza de la programación funcional

La programación funcional no trata solo de evitar bucles `for`. Trata de **Composición**: construir sistemas robustos y complejos ensamblando pequeñas funciones puras como si fueran bloques de Lego, tratándolas como el **valor** más importante del lenguaje.

¡Gracias!

Hagamos ejercicios en 4Clojure.